

CISP 400

Object Oriented Programming in C++

Dr. Fowler



F O L S O M L A K E C O L L E G E
EL DORADO CENTER | RANCHO CORDOVA CENTER

Last Time

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

**7.2 Object-Oriented
Programming**

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private Member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented
Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private Member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

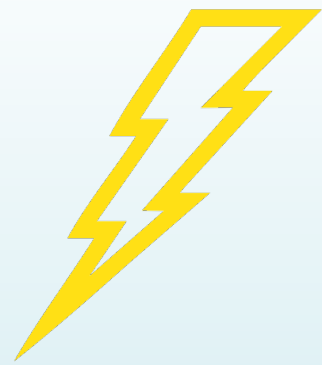
8.13 Arrays of Objects

Theme for Today

“Initializing Classes”

A large, stylized yellow lightning bolt graphic is positioned diagonally across the center of the slide. It has a jagged, stepped appearance with multiple sharp turns. The text "Lightning Review" is overlaid on the lightning bolt.

Lightning Review



Lightning Review Questions

Great time to get Peer Review Partners

- How do we get data into and out of an object?
- What other questions to you have for me?

Programming Exercise

- Modify the Car class, adding setters and getters for model and year.
- Pass a value as an argument for the model setter.
- Using cin for the year setter and prompt the user for a value.

```

1  //thirdObjInClassA1.cpp
2  #include <iostream>
3  using namespace std;
4
5  // Define the class
6  class Car {
7  private:
8      string make;
9      string model;
10     int year;
11 public:
12     void start() {
13         cout << "Car started!" << endl;
14     }
15     void SetMake() {
16         make = "Ford";
17     }
18     string GetMake() {
19         return make;
20     }
21     void SetModel(string m) {
22         model = m;
23     }
24     string GetModel() {
25         return " " + model + " ";
26     }
27     void SetYear() {
28         cout << "Enter the year: ";
29         cin >> year;
30     }
31     int GetYear() {
32         return year;
33     }
34 };
35
36 int main() {
37     // Instantiate an object of the class
38     Car myCar; // Create an object of type Car
39
40     // Use the object
41     myCar.SetMake();
42     myCar.SetModel("Mustang");
43     myCar.SetYear();
44     cout << myCar.GetMake() << myCar.GetModel() << myCar.GetYear() << endl;
45     myCar.start(); // Call the class method
46
47     return 0;
48 }

```

```

1  //thirdObjInClassA1.cpp
2  #include <iostream>
3  using namespace std;
4
5  // Define the class
6  class Car {
7  private:
8      string make;
9      string model;
10     int year;
11 public:
12     void start() {
13         cout << "Car started!" << endl;
14     }
15     ...
21     void SetModel(string m) {
22         model = m;
23     }
24     string GetModel() {
25         return " " + model + " ";
26     }
27     ...
34 };
35
36 int main() {
37     // Instantiate an object of the class
38     Car myCar; // Create an object of type Car
39
40     // Use the object
41     myCar.SetMake();
42     myCar.SetModel("Mustang");
43     myCar.SetYear();
44     cout << myCar.GetMake() << myCar.GetModel() << myCar.GetYear() << endl;
45     myCar.start(); // Call the class method
46
47     return 0;
48 }

```

```
1 //thirdObjInClassA1.cpp
2 #include <iostream>
3 using namespace std;
4
5 // Define the class
6 class Car {
7 private:
8     string make;
9     string model;
10    int year;
11 public:
12    void start() {
13        cout << "Car started!" << endl;
14    }
15    ...
21    ...
27    void SetYear() {
28        cout << "Enter the year: ";
29        cin >> year;
30    }
31    int GetYear() {
32        return year;
33    }
34 };
35
36 int main() {
37     // Instantiate an object of the class
38     Car myCar; // Create an object of type Car
39
40     // Use the object
41     myCar.SetMake();
42     myCar.SetModel("Mustang");
43     myCar.SetYear();
44     cout << myCar.GetMake() << myCar.GetModel() << myCar.GetYear() << endl;
45     myCar.start(); // Call the class method
46
47     return 0;
48 }
```

```

1  //thirdObjInClassA1.cpp
2  #include <iostream>
3  using namespace std;
4
5  // Define the class
6  class Car {
7  private:
8      string make;
9      string model;
10     int year;
11 public:
12     void start() {
13         cout << "Car started!" << endl;
14     }
15     void SetMake() {
16         make = "Ford";
17     }
18     string GetMake() {
19         return make;
20     }
21     void SetModel(string m) {
22         model = m;
23     }
24     string GetModel() {
25         return " " + model + " ";
26     }
27     void SetYear() {
28         cout << "Enter the year: ";
29         cin >> year;
30     }
31     int GetYear() {
32         return year;
33     }
34 };
35
36 int main() {
37     // Instantiate an object of the class
38     Car myCar; // Create an object of type Car
39
40     // Use the object
41     myCar.SetMake();
42     myCar.SetModel("Mustang");
43     myCar.SetYear();
44     cout << myCar.GetMake() << myCar.GetModel() << myCar.GetYear() << endl;
45     myCar.start(); // Call the class method
46
47     return 0;
48 }

```

```

Enter the year: 2022
Ford Mustang 2022
Car started!

```

...Program finished with exit code 0
Press ENTER to exit console.

Relationships

- As you know, objects don't work in isolation. They have **relationships** with each other. We can exploit these relationships to make our code more modular, reliable and simpler.
- There are a number of relationships . . .

Relationship Type	Relationship Example	Key Characteristic
Association	Student ↔ Course	General uses relationship
Aggregation	Library → Book	Weak ownership (has-a)
Composition	Car → Engine	Strong ownership (part-of)
Inheritance	Dog → Animal	Hierarchical (is-a)
Dependency	Printer → Document	Temporary use
Realization	Bird → Flyable	Implements behavior
Friendship	Matrix ↔ Calculator	Explicit access granting

We will cover some of these individually.

Relationship Type	Relationship Example	Key Characteristic
Association	Student ↔ Course	General uses relationship
Aggregation	Library → Book	Weak ownership (has-a)
Composition	Car → Engine	Strong ownership (part-of)
Inheritance	Dog → Animal	Hierarchical (is-a)
Dependency	Printer → Document	Temporary use
Realization	Bird → Flyable	Implements behavior
Friendship	Matrix ↔ Calculator	Explicit access granting

Dependency Relationship

- The “loosest” form of class relationship.
- Means: Class A uses Class B temporarily.
- AKA a “uses-a” relationship.

Coding Characteristics

- Short-lived interaction.
- Class A depends on class B, but is not stored in class A.

Example 1

```
1  #include <iostream>
2  using namespace std;
3
4  class Document {
5  public:
6      string getContent() const {
7          return "This is the document content.";
8      }
9  };
10
11 class Printer {
12 public:
13     void printDocument(const Document& doc) { // *** Dependency ***
14         cout << "Printing: " << doc.getContent() << endl;
15     }
16 };
17
18 int main() {
19     Document myDoc;
20     Printer myPrinter;
21
22     myPrinter.printDocument(myDoc);           // Uses Document temporarily
23 }
24
25 /*
26 Printer needs to know something about the documents coming in.
27 That's why we use a reference.
28 */
```

Example 2

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // ----- Customer Class -----
6  class Customer {
7  private:
8      string name;
9
10 public:
11     Customer(const string& name) : name(name) {}
12
13     string getName() const {
14         return name;
15     }
16 };
17
18 // ----- Order Class -----
19 class Order {
20 public:
21     void printCustomerInfo(const Customer& cust) {
22         cout << "Processing order for customer: " << cust.getName() << endl;
23     }
24
25     void sendThankYou(const Customer& cust) {
26         cout << "Thank you, " << cust.getName() << ", for your order!" << endl;
27     }
28 };
29
30 // ----- Main Function -----
31 int main() {
32     Customer c1("Alice");
33     Customer c2("Bob");
34
35     Order order;
36
37     order.printCustomerInfo(c1);    // Order "uses" Customer (dependency)
38     order.sendThankYou(c2);        // Another dependency use
39     order.printCustomerInfo(Customer("Charlie")); // Also works, temp object
40
41     return 0;
42 }

```



```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // ----- Customer Class -----
6  class Customer {
7  private:
8      string name;
9
10 public:
11     Customer(const string& name) : name(name) {}
12
13     string getName() const {
14         return name;
15     }
16 };
17
18 // ----- Order Class -----
19 class Order {
20 public:
21     void printCustomerInfo(const Customer& cust) {
22         cout << "Processing order for customer: " << cust.getName() << endl;
23     }
24
25     void sendThankYou(const Customer& cust) {
26         cout << "Thank you, " << cust.getName() << ", for your order!" << endl;
27     }
28 };
29
30 // ----- Main Function -----
31 int main() {
32     Customer c1("Alice");
33     Customer c2("Bob");
34
35     Order order;
36
37     order.printCustomerInfo(c1);    // Order "uses" Customer (dependency)
38     order.sendThankYou(c2);        // Another dependency use
39     order.printCustomerInfo(Customer("Charlie")); // Also works, temp object
40
41     return 0;
42 }

```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // ----- Customer Class -----
6  class Customer {
7  private:
8      string name;
9
10 public:
11     Customer(const string& name) : name(name) {}
12
13     string getName() const {
14         return name;
15     }
16 };
17

```



```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // ----- Customer Class -----
6  class Customer {
7  private:
8      string name;
9
10 public:
11     Customer(const string& name) : name(name) {}
12
13     string getName() const {
14         return name;
15     }
16 };
17
18 // ----- Order Class -----
19 class Order {
20 public:
21     void printCustomerInfo(const Customer& cust) {
22         cout << "Processing order for customer: " << cust.getName() << endl;
23     }
24
25     void sendThankYou(const Customer& cust) {
26         cout << "Thank you, " << cust.getName() << ", for your order!" << endl;
27     }
28 };
29
30 // ----- Main Function -----
31 int main() {
32     Customer c1("Alice");
33     Customer c2("Bob");
34
35     Order order;
36
37     order.printCustomerInfo(c1); // Order "uses" Customer (dependency)
38     order.sendThankYou(c2);      // Another dependency use
39     order.printCustomerInfo(Customer("Charlie")); // Also works, temp object
40
41     return 0;
42 }

```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // ----- Customer Class -----
6  class Customer {
7  private:
8      string name;
9
10 public:
11     Customer(const string& name) : name(name) {}
12
13     string getName() const {
14         return name;
15     }
16 };
17
18 // ----- Order Class -----
19 class Order {
20 public:
21     void printCustomerInfo(const Customer& cust) {
22         cout << "Processing order for customer: " << cust.getName() << endl;
23     }
24
25     void sendThankYou(const Customer& cust) {
26         cout << "Thank you, " << cust.getName() << ", for your order!" << endl;
27     }
28 };
29
30 // ----- Main Function -----
31 int main() {
32     Customer c1("Alice");
33     Customer c2("Bob");
34
35     Order order;
36
37     order.printCustomerInfo(c1); // Order "uses" Customer (dependency)
38     order.sendThankYou(c2);      // Another dependency use
39     order.printCustomerInfo(Customer("Charlie")); // Also works, temp object
40
41     return 0;
42 }

```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // ----- Customer Class -----
6  class Customer {
7  private:
8      string name;
9
10 public:
11     Customer(const string& name) : name(name) {}
12
13     string getName() const {
14         return name;
15     }
16 };
17
18 // ----- Order Class -----
19 class Order {
20 public:
21     void printCustomerInfo(const Customer& cust) {
22         cout << "Processing order for customer: " << cust.getName() << endl;
23     }
24
25     void sendThankYou(const Customer& cust) {
26         cout << "Thank you, " << cust.getName() << ", for your order!" << endl;
27     }
28 };
29
30 // ----- Main Function -----
31 int main() {
32     Customer c1("Alice");
33     Customer c2("Bob");
34
35     Order order;
36
37     order.printCustomerInfo(c1);    // Order "uses" Customer (dependency)
38     order.sendThankYou(c2);        // Another dependency use
39     order.printCustomerInfo(Customer("Charlie")); // Also works, temp object
40
41     return 0;
42 }

```

Output

```

Processing order for customer: Alice
Thank you, Bob, for your order!
Processing order for customer: Charlie

```

```

=== Code Execution Successful ===

```

- Do we always need to use reference variables?

No

```
1 class Logger {  
2 public:  
3     void logMessage(string message) {           // *** Passed by value  
4         cout << "[LOG]: " << message << endl;  
5     }  
6 };  
7
```

- Technically this is a dependency relationship.
- But because strings are a standard type, nobody thinks of it that way.
- Nor do we need a reference because strings are built in.

Example 3

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // --- User (returned by another class) ---
6  class User {
7  private:
8      string username;
9  public:
10     User(string name) : username(name) {}
11     string getUsername() const { return username; }
12 };
13
14 // --- UserDatabase (returns User) ---
15 class UserDatabase {
16 public:
17     User getUserById(int id) {
18         // Simulated lookup
19         return User("user_" + to_string(id));
20     }
21 };
22
23 // --- Main ---
24 int main() {
25     UserDatabase db;
26     User u = db.getUserById(42); // ➡ Dependency via return type
27     cout << "Fetched user: " << u.getUsername() << endl;
28
29     return 0;
30 }

```

Output

Fetches user: user_42

=== Code Execution Successful ===

Exercise

Desk Check

Exercise Goals

- Review of code syntax
- Check understanding of syntax

Desk Check Exercise

- Pair up
- Manually determine the output of the following code?
DO NOT TYPE THIS IN AND EXECUTE IT.
- We'll spend about 10 minutes on this, then go over it.

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Speaker {
6  public:
7      void sayHello(const string& name) {
8          cout << "Hello, " << name << "!" << endl;
9      }
10 };
11
12 class Greeter {
13 public:
14     void greetUser(const string& userName) {
15         Speaker s; // 🖱️ Local variable: dependency
16         s.sayHello(userName);
17     }
18 };
19
20 int main() {
21     Greeter g;
22     g.greetUser("Taylor");
23     return 0;
24 }
```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Speaker {
6  public:
7      void sayHello(const string& name) {
8          cout << "Hello, " << name << "!" << endl;
9      }
10 };
11
12 class Greeter {
13 public:
14     void greetUser(const string& userName) {
15         Speaker s; // 🖱️ Local variable: dependency
16         s.sayHello(userName);
17     }
18 };
19
20 int main() {
21     Greeter g;
22     g.greetUser("Taylor");
23     return 0;
24 }

```

Hello, Taylor!

=== Code Execution Successful ===

8.13 Arrays of Objects

- Can we create arrays of objects or structs?
- Absolutely - they are just another variable type!

- Let's go ahead and build one on the stack ...


```
1 //simpObjArr.cpp
5
6 using namespace std;
7
8 // Simple Book class definition
9 class Book {
10     string title;    // private: is default
11     string author;
12
13 public:
14     // Constructor
15     Book(string t = "Untitled", string a = "Unknown") : title(t), author(a) {}
16
17     // Method to display book information
18     void displayInfo() const {
19         cout << "Title: " << title << ", Author: " << author << endl;
20     }
21 };
22
23 int main() {
24     // Declaring an array of 3 Book objects
25     Book books[3] = {
26         Book("1984", "George Orwell"),
27         Book("To Kill a Mockingbird", "Harper Lee"),
28         Book("The Catcher in the Rye", "J.D. Salinger")
29     };
30
31     // Loop through the array and display information about each book
32     for (int i = 0; i < 3; ++i)
33         books[i].displayInfo();
34
35     return 0;
36 }
```



```

1 //simpObjArr.cpp
5
6 using namespace std;
7
8 // Simple Book class definition
9 class Book {
10     string title;    // private: is default
11     string author;
12
13 public:
14     // Constructor
15     Book(string t = "Untitled", string a = "Unknown") : title(t), author(a) {}
16
17     // Method to display book information
18     void displayInfo() const {
19         cout << "Title: " << title << ", Author: " << author << endl;
20     }
21 };
22
23 int main() {
24     // Declaring an array of 3 Book objects
25     Book books[3] = {
26         Book("1984", "George Orwell"),
27         Book("To Kill a Mockingbird", "Harper Lee"),
28         Book("The Catcher in the Rye", "J.D. Salinger")
29     };
30
31     // Loop through the array and display information about each book
32     for (int i = 0; i < 3; ++i)
33         books[i].displayInfo();
34
35     return 0;
36 }

```

```

Title: 1984, Author: George Orwell
Title: To Kill a Mockingbird, Author: Harper Lee
Title: The Catcher in the Rye, Author: J.D. Salinger

```

...Program finished with exit code 0

- What's happening here?

```
13 public:
14     // Constructor
15     Book(string t = "Untitled", string a = "Unknown") : title(t), author(a) {}
16
```

- What's happening here?

```
13 public:
14     // Constructor
15     Book(string t = "Untitled", string a = "Unknown") : title(t), author(a) {}
16
```



This is called a **member initialization list**.

It's a shorthand way of passing data in from a constructor.

The private variable *title* is used to declare variable *t*, and *t* automatically passes the data to *title* when read in with the constructor.

Empty parentheses because no explicit assignment is needed.



Ok,
But now we have
the problem of
memory on the
stack again!



Ok,
But now we have
the problem of
memory on the
stack again!

Can we make dynamic arrays of objects?

```
1 //simpObjArrHeap.cpp
5
6 class Book {
7     string title;
8     string author;
9 public:
10     Book(string t = "Untitled", string a = "Unknown") : title(t), author(a) {}
11
12     void displayInfo() const {
13         cout << "Title: " << title << ", Author: " << author << endl;
14     }
15 };
16
17 int main() {
18     // Dynamically allocating an array of 3 Book objects on the heap
19     Book* books = new Book[3]{
20         Book("1984", "George Orwell"),
21         Book("To Kill a Mockingbird", "Harper Lee"),
22         Book("The Catcher in the Rye", "J.D. Salinger")
23     };
24
25     // Loop through the array and display information about each book
26     for (int i = 0; i < 3; ++i) {
27         books[i].displayInfo();
28     }
29
30     // Free the allocated memory on the heap
31     delete[] books;
32
33     return 0;
34 }
```



```

1 //simpObjArrHeap.cpp
5
6 class Book {
7     string title;
8     string author;
9 public:
10     Book(string t = "Untitled", string a = "Unknown") : title(t), author(a) {}
11
12     void displayInfo() const {
13         cout << "Title: " << title << ", Author: " << author << endl;
14     }
15 };
16
17 int main() {
18     // Dynamically allocating an array of 3 Book objects on the heap
19     Book* books = new Book[3]{
20         Book("1984", "George Orwell"),
21         Book("To Kill a Mockingbird", "Harper Lee"),
22         Book("The Catcher in the Rye", "J.D. Salinger")
23     };
24
25     // Loop through the array and display information about each book
26     for (int i = 0; i < 3; ++i) {
27         books[i].displayInfo();
28     }
29
30     // Free the allocated memory on the heap
31     delete[] books;
32
33     return 0;
34 }

```

```

Title: 1984, Author: George Orwell
Title: To Kill a Mockingbird, Author: Harper Lee
Title: The Catcher in the Rye, Author: J.D. Salinger

```

...Program finished with exit code 0

- We constructed an array of objects on the heap.
- Ok, for simple situations.
- We will run into limitations later.

- In the meantime, let's make this array dynamic ...

```

1 //DynObjArrHeap.cpp
5
6 class Book {
14 };
15
16 Book* resizeArr(Book*, int, int);
17
18 int main() {
19     Book* books = new Book[3]{
20         Book("1984", "George Orwell"),
21         Book("To Kill a Mockingbird", "Harper Lee"),
22         Book("The Catcher in the Rye", "J.D. Salinger")
23     };
24
25     // Loop through the array and display information about each book
26     for (int i = 0; i < 3; ++i) books[i].displayInfo();
27     cout << "\nResizing\n";
28
29     books = resizeArr(books, 3, 5); // new array size here
30     books[3] = Book("Design Patterns", "Gamma");
31     books[4] = Book();
32
33     for (int i = 0; i < 5; ++i) books[i].displayInfo();
34
35     delete[] books;
36
37     return 0;
38 }
39 Book* resizeArr(Book* oldArr, int sOld, int sNew) {
40     Book* pTmp = new Book[sNew];
41     for (int i = 0; i < sOld; i++) pTmp[i] = oldArr[i];
42     delete [] oldArr;
43     return pTmp;
44 }

```

```

1 //DynObjArrHeap.cpp
5
6 class Book {
14 };
15
16 Book* resizeArr(Book*, int, int);
17
18 int main() {
19     Book* books = new Book[3]{
20         Book("1984", "George Orwell"),
21         Book("To Kill a Mockingbird", "Harper Lee"),
22         Book("The Catcher in the Rye", "J.D. Salinger")
23     };
24
25     // Loop through the array and display information about each book
26     for (int i = 0; i < 3; ++i) books[i].displayInfo();
27     cout << "\nResizing\n";
28
29     books = resizeArr(books, 3, 5); // new array size here
30     books[3] = Book("Design Patterns", "Gamma");
31     books[4] = Book();
32
33     for (int i = 0; i < 5; ++i)
34         books[i].displayInfo();
35     delete[] books;
36
37     return 0;
38 }
39 Book* resizeArr(Book* oldArr, int sNew, int sOld) {
40     Book* pTmp = new Book[sNew];
41     for (int i = 0; i < sOld; i++)
42         pTmp[i] = *oldArr[i];
43     delete [] oldArr;
44     return pTmp;
45 }

```

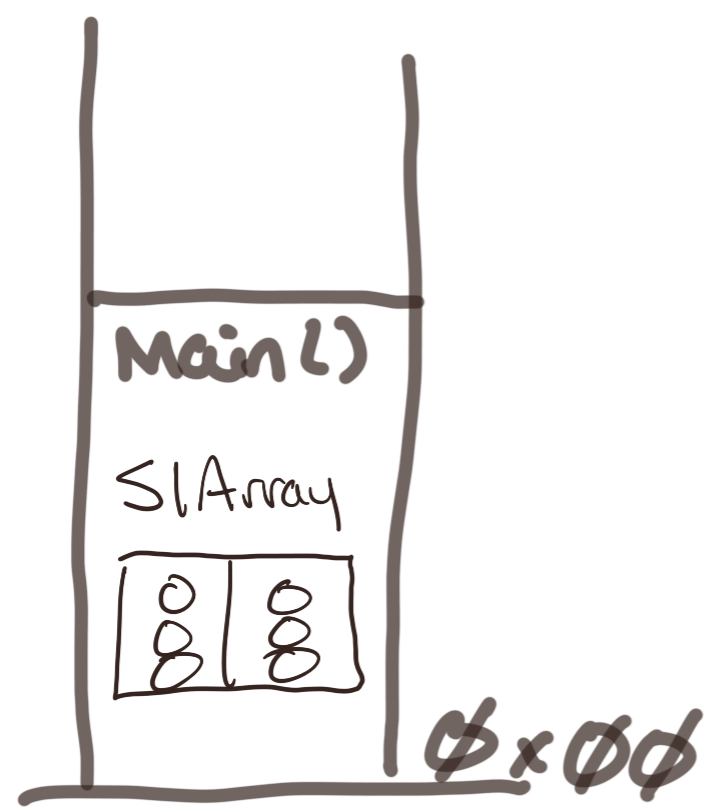
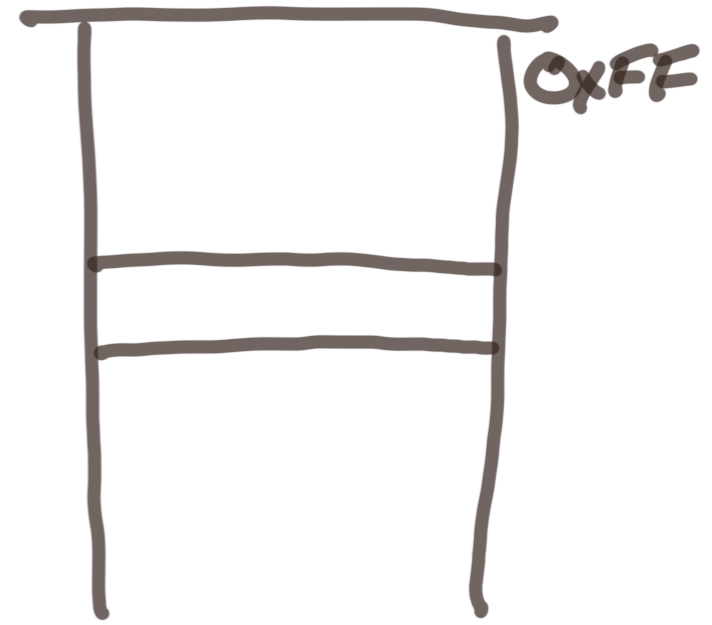
Title: 1984, Author: George Orwell
 Title: To Kill a Mockingbird, Author: Harper Lee
 Title: The Catcher in the Rye, Author: J.D. Salinger

 Resizing
 Title: 1984, Author: George Orwell
 Title: To Kill a Mockingbird, Author: Harper Lee
 Title: The Catcher in the Rye, Author: J.D. Salinger
 Title: Design Patterns, Author: Gamma
 Title: Untitled, Author: Unknown

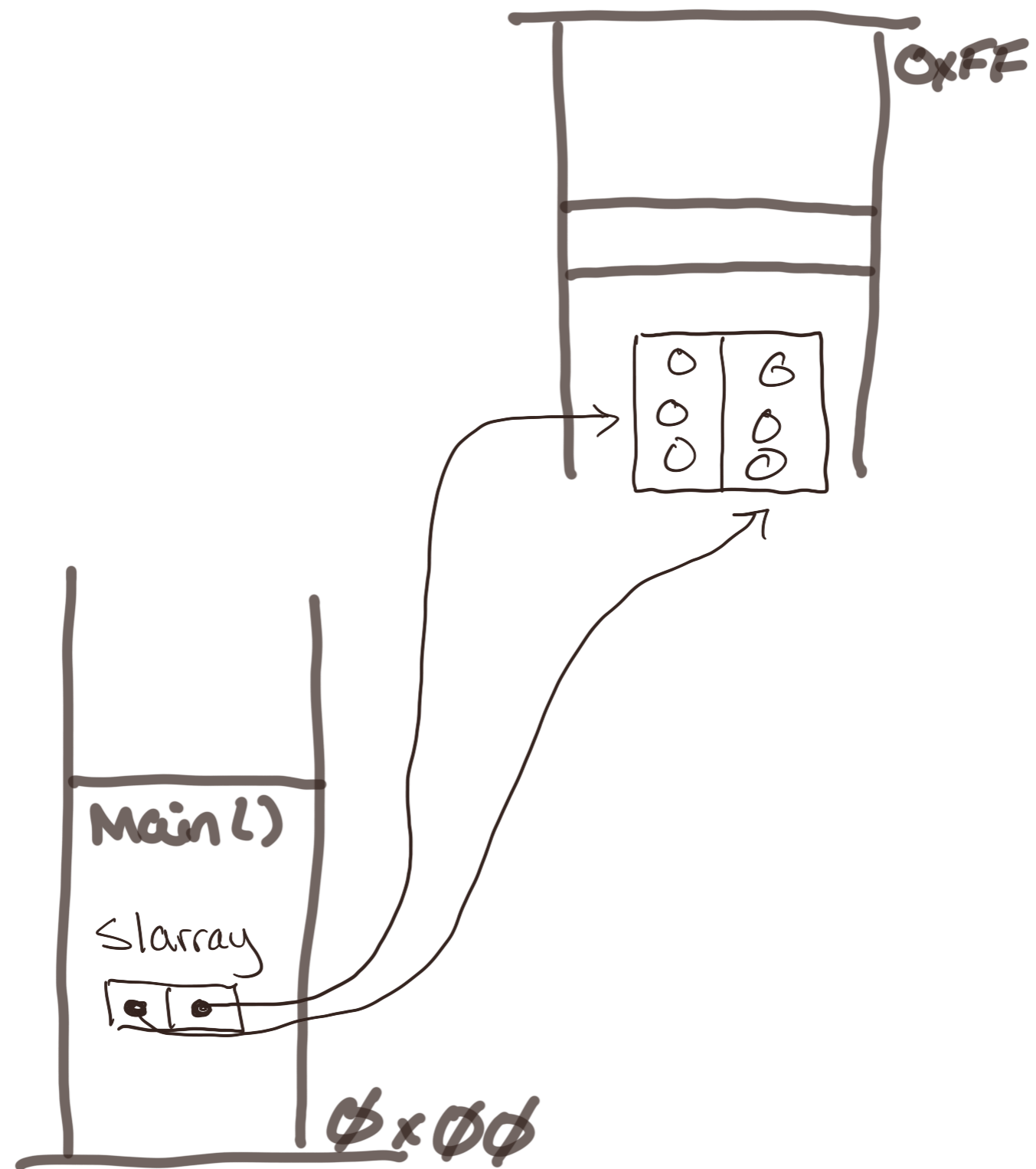
...Program finished with exit code 0

C++ Ninja Solution

```
1 // ds1a0.cpp -- demo of array of stoplight class
2 #include <iostream>
3 using namespace std;
4
5 class StopLight {
6 public:
7     enum Light {RED, YELLOW, GREEN};
8     Light steadyLight;
9     StopLight(){ steadyLight = YELLOW; };
10    StopLight(Light lightColor){ steadyLight = lightColor;};
11    void Cycle();
12 };
13 void StopLight::Cycle() {
14     switch (steadyLight){
15         case RED: cout << "Red\n"; break;
16         case YELLOW: cout << "Yellow\n"; break;
17         case GREEN: cout << "Green\n"; break;
18     }
19 }
20
21 int main(){
22     StopLight slArray[2];
23     slArray[0].Cycle();
24     slArray[1].Cycle();
25     cout << "\nCreate objects.\n\n";
26
27     slArray[0] = StopLight(StopLight::RED);
28     slArray[1] = StopLight(StopLight::YELLOW);
29
30     slArray[0].Cycle();
31     slArray[1].Cycle();
32
33     return 0;
34 }
```



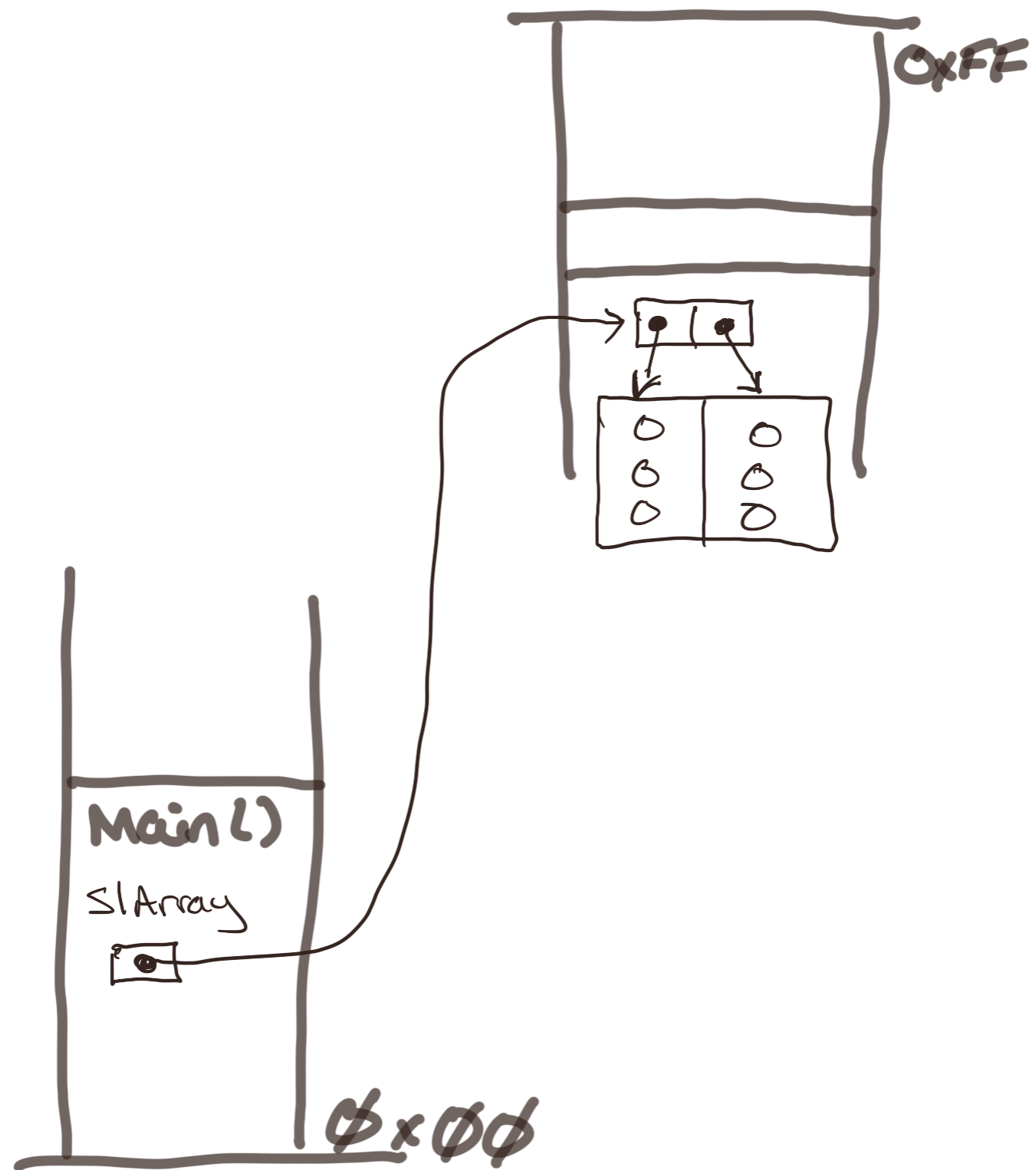

```
1  // ds1a1.cpp -- demo of array of stoplight class pointers
2  #include <iostream>
3  using namespace std;
4
5  class StopLight {
6  public:
7      enum Light {RED, YELLOW, GREEN};
8      Light steadyLight;
9      StopLight(){ steadyLight = YELLOW; };
10     StopLight(Light lightColor){ steadyLight = lightColor;};
11     void Cycle();
12 };
13 void StopLight::Cycle() {
14     switch (steadyLight){
15     case RED: cout << "Red\n"; break;
16     case YELLOW: cout << "Yellow\n"; break;
17     case GREEN: cout << "Green\n"; break;
18     }
19 }
20
21 int main(){
22     StopLight* slArray[2];
23     // slArray[0]->Cycle();    // warning w/ seg fault if run
24     slArray[0] = new StopLight(StopLight::RED);
25     slArray[1] = new StopLight(StopLight::YELLOW);
26
27     slArray[0]->Cycle();
28     slArray[1]->Cycle();
29
30
31     return 0;
32 }
```




```

1  // ds1a2.cpp -- dynamic array of stoplight class pointers
2  #include <iostream>
3  using namespace std;
4
5  class StopLight {
6  public:
7      enum Light {RED, YELLOW, GREEN};
8      Light steadyLight;
9      StopLight(){ steadyLight = YELLOW; };
10     StopLight(Light lightColor){ steadyLight = lightColor;};
11     void Cycle();
12 };
13 void StopLight::Cycle() {
14     switch (steadyLight){
15     case RED: cout << "Red\n"; break;
16     case YELLOW: cout << "Yellow\n"; break;
17     case GREEN: cout << "Green\n"; break;
18     }
19 }
20
21 int main(){
22     StopLight** slArray = new StopLight*[2];
23     // slArray[0]->Cycle();    // warning w/ seg fault if run
24     slArray[0] = new StopLight(StopLight::RED);
25     slArray[1] = new StopLight(StopLight::YELLOW);
26
27     slArray[0]->Cycle();
28     slArray[1]->Cycle();
29
30
31     return 0;
32 }

```



Assertions

- We have the debugging process down.
- we need a mechanism to detect incorrect states.

- IF statements
 - Hard to turn off.
 - Make our code bigger when not used.



Assertions

- Look like program code.
- Can turn off and on.
- Compile out of the program when done.

```
1  #include <iostream>
2  #include <cassert>
3  // uncomment to disable assert()
4  // #define NDEBUG
5  using namespace std;
6
7  // Function Prototype
8  int Doubler(int);
9
10 int main() {
11     int i = 1;
12
13     cout << Doubler(i) << endl;
14 }
15
16 int Doubler(int i) {
17     assert(i > 0);           // confirm good data
18     int x = i * i;
19
20     assert(x == i * i);     // confirm good result
21     return x;
22 }
```


Exercise

Desk Check

Exercise Goals

- Review of code syntax
- Check understanding of syntax

Desk Check Exercise

- Pair up
- Manually determine the output of the following code?
DO NOT TYPE THIS IN AND EXECUTE IT.
- We'll spend about 10 minutes on this, then go over it.

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Message {
6  private:
7      string text;
8  public:
9      Message(const string& t) : text(t) {}
10     string getText() const { return text; }
11 };
12
13 class Messenger {
14 public:
15     void deliverMessage(const Message& msg) {
16         cout << "Delivering: " << msg.getText() << endl;
17     }
18 };
19
20 class Communicator {
21 public:
22     void start() {
23         Message m("Mission Start");           // Local dependency
24         Messenger courier;                     // Another local dependency
25         courier.deliverMessage(m);
26     }
27 };
28
29 int main() {
30     Communicator c;
31     c.start();
32     return 0;
33 }
```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Message {
6  private:
7      string text;
8  public:
9      Message(const string& t) : text(t) {}
10     string getText() const { return text; }
11 };
12
13 class Messenger {
14 public:
15     void deliverMessage(const Message& msg) {
16         cout << "Delivering: " << msg.getText() << endl;
17     }
18 };
19
20 class Communicator {
21 public:
22     void start() {
23         Message m("Mission Start");           // Local dependency
24         Messenger courier;                     // Another local dependency
25         courier.deliverMessage(m);
26     }
27 };
28
29 int main() {
30     Communicator c;
31     c.start();
32     return 0;
33 }

```

Delivering: Mission Start

=== Code Execution Successful ===

Programming Exercise

Activity

- Create a class Message with:
 - A string content
 - A constructor that sets the content
 - A method string getContent() const
- Create a class Notifier with:
 - A method void sendNotification(const Message& msg)
 - It should print: "Notification: <message content>"
- In main():
 - Create two Message objects (e.g., "You have mail", "Your package has shipped")
 - Create a Notifier object
 - Use Notifier to send both messages.

```
#include <iostream>
#include <string>
using namespace std;

// 1. Define Message class here

// 2. Define Notifier class here

// 3. Main function to demonstrate dependency
int main() {
    // Create Message objects

    // Create Notifier

    // Use Notifier to send messages

    return 0;
}
```

```
Notification: You have mail
Notification: Your package has shipped
```




What's this section
about?

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented
Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects

What's Next?

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented
Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented
Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects